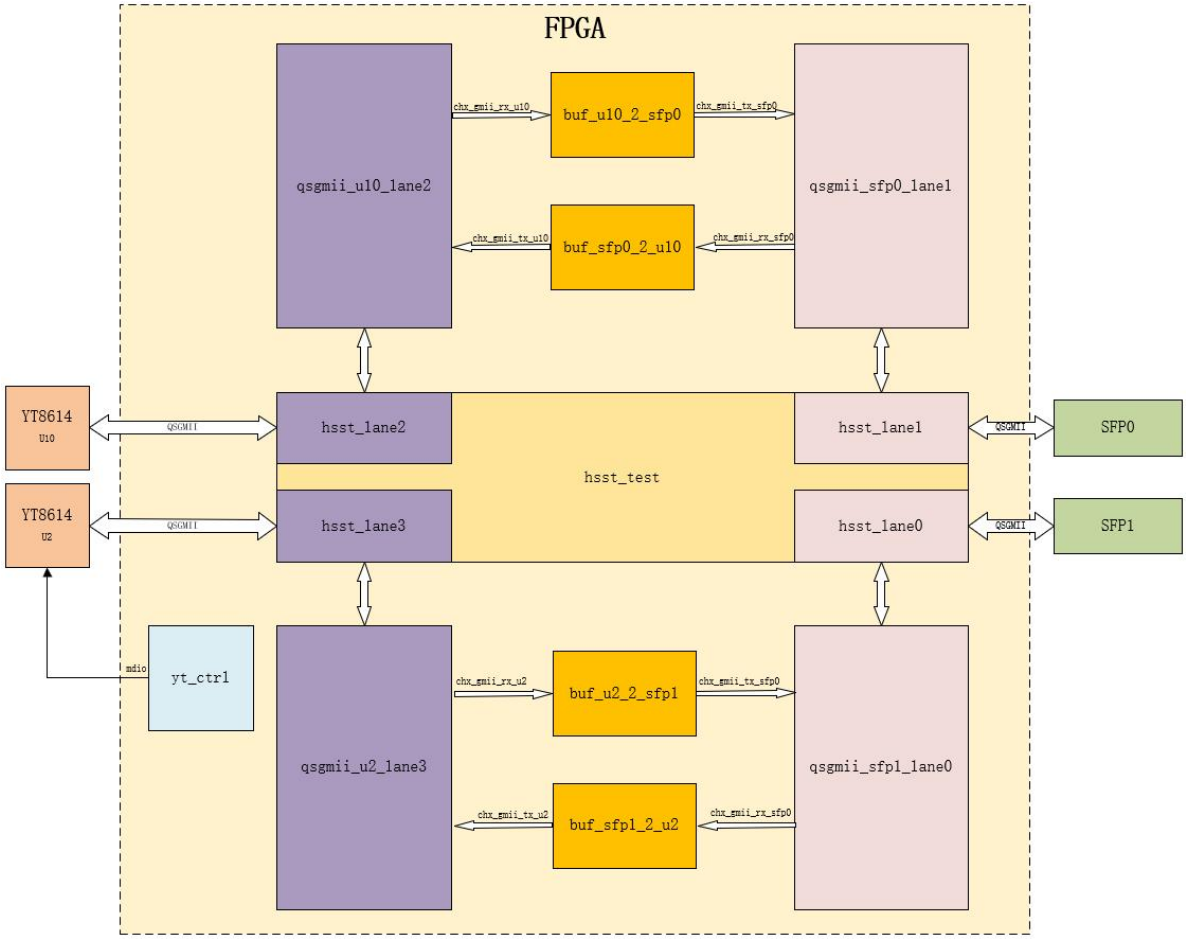
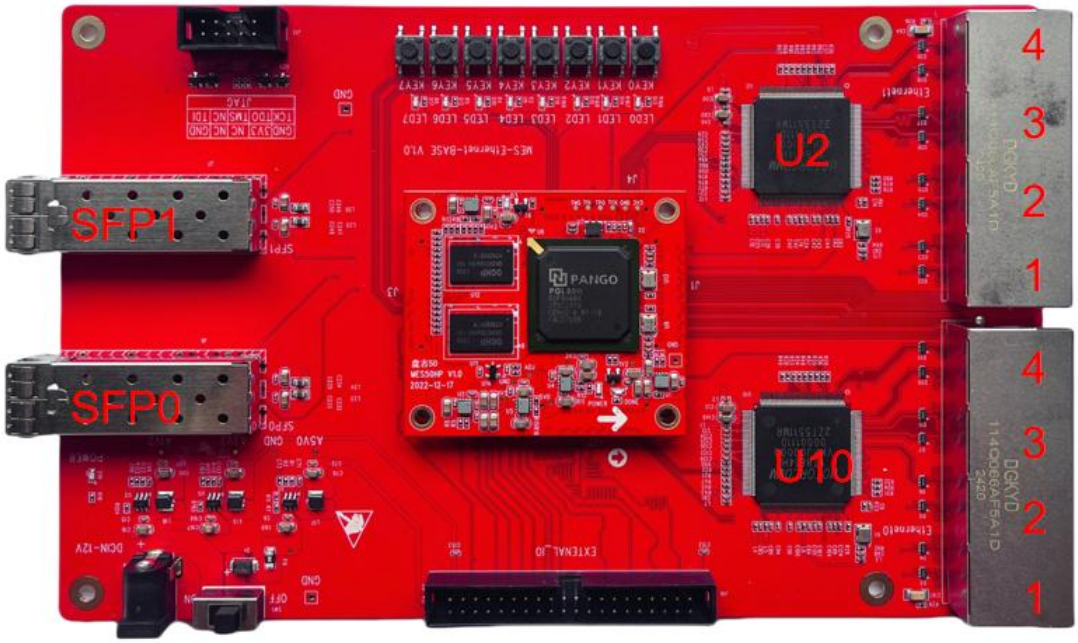
网口底板 demo 说明

软件版本：PDS2022_1
功能：可实现八路网口通过光纤接口级联完成自环。

一、工程框架



YT8614 芯片将 4 路网口数据打包成 QSGMII 的格式再通过 HSST 的 lane2 和 lane3 将差分串行数据给到 QSGMII IP 核，将 QSGMII 协议的数据解析出 4 路 GMII 数据给到用户，之后将数据给到 buf 模块，buf 模块主要将 GMII 数据跨时钟传输到 SFP 口的 QSGMII IP 的时钟下，SFP 口的 QSGMII IP 的差分数据再给到 HSST 的 lane0 和 lane1，让数据通过 SFP 口完成收发。



工程具体说明：PC1 网口插入下方 4 路网口的 1 口，之后网口数据会通过 U10(YT8614)，打包成 QSGMII 协议的数据，然后 SFP0 和 SFP1 通过光纤连接，下方 4 路网口的 1 口的数据会送到 SFP1 中，SFP1 中的数据再给到 U2(yt8614)解析成电口数据给到上方 4 路网口的 1 口，该口再接 PC2，即完成自环。

此时 PC1 和 PC2 可以互相 ping 通，即表示自环成功。

注意网口要一一对应，即下方的网口的 1、2、3、4 和上方的网口的 1、2、3、4 要对应起来才能完成通路。

二、代码模块说明

2.1 QSGMII IP 说明

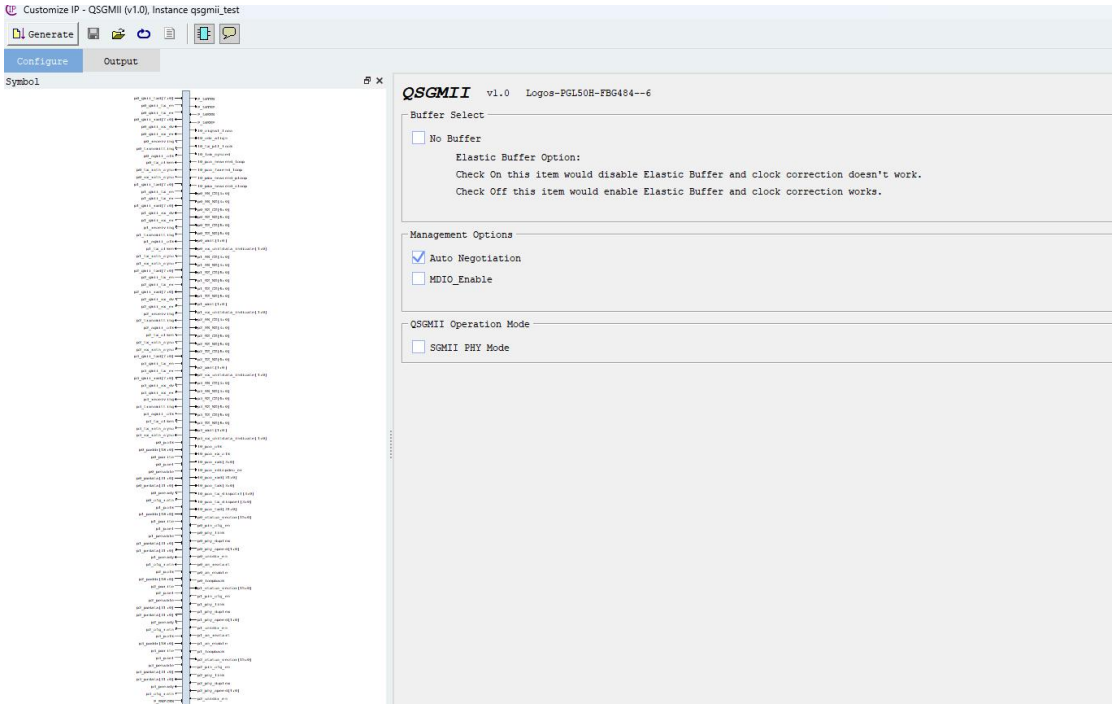
QSGMII IP 的配置：

注：需要先安装该 IP，具体安装步骤与 DDR3 等 IP 的安装一致。

本次 DEMO 工程也对 QSGMII IP 生成的顶层进行了修改，将 HSST 接口开放。要完成移植，请详细对照该顶层进行移植，否则会出现 HSST 资源不够用的情况。

QSGMII 的端口说明可以查看 IP 手册以及例程。

QSGMII IP 的配置如下：



该 IP 配置界面简介，勾选 Auto Negotiation 即可。

2.2 yt8614_ctrl 模块

```
module yt8614_ctrl#(
 parameter RSTN_20MS    = 2000_000          ,
    parameter RSTN_120MS   = 12000_000         ,
    parameter MDC_120MS    = 1000_000
)(
    input               i_clk                  ,
    input               i_rst_n                ,
    output              reset_n_out            ,
    output              reset_n_done           ,
    output              mdc                    ,
    inout               mdio
);
//////////////////////////////////////////////////////////////////////
/
reset_n_delay#(
    .RSTN_120MS      (RSTN_120MS      ),
    .RSTN_20MS       (RSTN_20MS       )
)u_reset_n_delay(
    .i_clk          (i_clk          ),// input
    .i_rst_n        (i_rst_n        ),// input
    .reset_n_out    (reset_n_out    ),// output
    .reset_n_done   (reset_n_done   ) // output
);
```

```
23. ////////////////////////////////////////////////////////////////
    /
24. wire                start           ;
25. wire [1:0]          opcode          ;
26. wire [4:0]          phy_addr        ;
27. wire [4:0]          reg_addr        ;
28. wire [15:0]         write_data      ;
29. wire [15:0]         read_data       /*synthesis PAP_MARK_DEBUG="1"*
    /;
30. wire                read_data_en    /*synthesis PAP_MARK_DEBUG="1"*
    /;
31. wire                ready           ;
32.
33. mdio_master_driver u_mdio_master_driver(
34.     .clk            (i_clk          ),// input 50M
35.     .rstn           (i_rst_n        ),// input
36.     .reset_n_done   (reset_n_done   ),// input
37.     .mdc            (mdc            ),// output
38.     .mdio           (mdio           ),// inout
39.     .start          (start          ),// input
40.     .opcode         (opcode         ),// input[1:0]
41.     .phy_addr       (phy_addr       ),// input[4:0]
42.     .reg_addr       (reg_addr       ),// input[4:0]
43.     .write_data     (write_data     ),// input[15:0]
44.     .read_data      (read_data      ),// output[15:0]
45.     .read_data_en   (read_data_en   ),// output
46.     .ready          (ready          ) // output
47. );
48. reg_ctrl u_reg_ctrl(
49.     .i_clk          (mdc            ),// input
50.     .reset_n_done   (reset_n_done   ),// input
51.     .ready          (ready          ),// input
52.     .start          (start          ),// output
53.     .opcode         (opcode         ),// output
54.     .phy_addr       (phy_addr       ),// output
55.     .reg_addr       (reg_addr       ),// output
56.     .write_data     (write_data     ),// output
57.     .read_data      (read_data      ),// input[15:0]
58.     .read_data_en   (read_data_en   ) // input
59. );
60. ////////////////////////////////////////////////////////////////
    /
61. endmodule
```

该模块主要通过 MDIO 接口完成对 TY8614 芯片的配置。该配置保持默认即可，

切勿改动。

2.3 led_test 模块

```
1.  module led_test(
2.      input           clk          ,
3.      input           rstn         ,
4.      input  [7:0]    done         ,
5.      input  [7:0]    link         ,
6.  //speed
7.      input  [1:0]    u2_ch0_speed ,
8.      input  [1:0]    u2_ch1_speed ,
9.      input  [1:0]    u2_ch2_speed ,
10.     input  [1:0]    u2_ch3_speed ,
11.     input  [1:0]    u10_ch0_speed,
12.     input  [1:0]    u10_ch1_speed,
13.     input  [1:0]    u10_ch2_speed,
14.     input  [1:0]    u10_ch3_speed,
15. //phy_led
16.     output [2:0]    u2_ch0_led   ,
17.     output [2:0]    u2_ch1_led   ,
18.     output [2:0]    u2_ch2_led   ,
19.     output [2:0]    u2_ch3_led   ,
20.     output [2:0]    u10_ch0_led  ,
21.     output [2:0]    u10_ch1_led  ,
22.     output [2:0]    u10_ch2_led  ,
23.     output [2:0]    u10_ch3_led  ,
24. //LED
25.     output [7:0]    led
26. );
27.
28. //reg and wire
29.
30.     reg [25:0] led_light_cnt    ;
31.     reg        led_status       ;
32.
33. //time counter
34.     always @(posedge clk)
35.     begin
36.         if(!rstn)
37.             led_light_cnt <=  26'd0;
38.         else if(led_light_cnt == 26'd9_999_999)
39.             led_light_cnt <=  26'd0;
40.         else
```

```
            led_light_cnt <=  led_light_cnt + 26'd1;
    end

//led status change
    always @(posedge clk)
    begin
        if(!rstn)
            led_status <=  1'b0;
        else if(led_light_cnt == 26'd9_999_999)
            led_status <=  ~led_status;
    end
//led
    assign led[0] = done[0];
    assign led[1] = done[1];
    assign led[2] = done[2];
    assign led[3] = done[3];
    assign led[4] = done[4];
    assign led[5] = done[5];
    assign led[6] = done[6];
    assign led[7] = done[7];
////////////////////////////////////////////////////////////////////////////
//////
assign u10_ch0_led  = (u10_ch0_speed==2'b10&&link[0])?{led_status,2'b01}:3'b
100;
assign u10_ch1_led  = (u10_ch1_speed==2'b10&&link[1])?{led_status,2'b01}:3'b
100;
assign u10_ch2_led  = (u10_ch2_speed==2'b10&&link[2])?{led_status,2'b01}:3'b
100;
assign u10_ch3_led  = (u10_ch3_speed==2'b10&&link[3])?{led_status,2'b01}:3'b
100;
assign u2_ch0_led   = (u2_ch0_speed==2'b10 &&link[4])?{led_status,2'b01}:3'b
100;
assign u2_ch1_led   = (u2_ch1_speed==2'b10 &&link[5])?{led_status,2'b01}:3'b
100;
assign u2_ch2_led   = (u2_ch2_speed==2'b10 &&link[6])?{led_status,2'b01}:3'b
100;
assign u2_ch3_led   = (u2_ch3_speed==2'b10 &&link[7])?{led_status,2'b01}:3'b
100;
////////////////////////////////////////////////////////////////////////////
//////
endmodule
```

该模块用来判断板卡上的 serdes 的链路是否正常，如果全部正常，板卡上 8 个 LED 灯均会点亮。done 信号是来自 hsst IP 的接收、发送完成信号。Link 信号是 FPGA 是否和 YT8614 完成建链的信号。

2.4 gmii_buf 模块

```
1.  module gmii_buf(
2.  //gmii_rx
3.      //ch0
4.      input                   ch0_gmii_rx_clk     ,
5.      input                   ch0_rx_rstn         ,
6.      input  [7:0]            ch0_gmii_rxd        ,
7.      input                   ch0_gmii_rx_dv      ,
8.      input                   ch0_gmii_rx_er      ,
9.      //ch1
10.     input                   ch1_gmii_rx_clk     ,
11.     input                   ch1_rx_rstn         ,
12.     input  [7:0]            ch1_gmii_rxd        ,
13.     input                   ch1_gmii_rx_dv      ,
14.     input                   ch1_gmii_rx_er      ,
15.     //ch2
16.     input                   ch2_gmii_rx_clk     ,
17.     input                   ch2_rx_rstn         ,
18.     input  [7:0]            ch2_gmii_rxd        ,
19.     input                   ch2_gmii_rx_dv      ,
20.     input                   ch2_gmii_rx_er      ,
21.     //ch3
22.     input                   ch3_gmii_rx_clk     ,
23.     input                   ch3_rx_rstn         ,
24.     input  [7:0]            ch3_gmii_rxd        ,
25.     input                   ch3_gmii_rx_dv      ,
26.     input                   ch3_gmii_rx_er      ,
27.
28. //gmii_tx
29.     //ch0
30.     input                   ch0_gmii_tx_clk     ,
31.     output  [7:0]           ch0_gmii_txd        ,
32.     output                  ch0_gmii_tx_en      ,
33.     output                  ch0_gmii_tx_er      ,
34.     //ch1
35.     input                   ch1_gmii_tx_clk     ,
36.     output  [7:0]           ch1_gmii_txd        ,
37.     output                  ch1_gmii_tx_en      ,
38.     output                  ch1_gmii_tx_er      ,
39.     //ch2
40.     input                   ch2_gmii_tx_clk     ,
41.     output  [7:0]           ch2_gmii_txd        ,
```

```
    output                      ch2_gmii_tx_en      ,
    output                      ch2_gmii_tx_er      ,
    //ch3
    input                       ch3_gmii_tx_clk     ,
    output  [7:0]               ch3_gmii_txd        ,
    output                      ch3_gmii_tx_en      ,
    output                      ch3_gmii_tx_er
);
////////////////////////////////////////////////////////////////////////////////
wire  [9:0]  wr_data_0 , wr_data_1,  wr_data_2,  wr_data_3 ;
wire         wr_en_0   , wr_en_1  ,  wr_en_2  ,  wr_en_3   ;
wire         rd_empty_0, rd_empty_1, rd_empty_2, rd_empty_3;
wire  [9:0]  rd_data_0 , rd_data_1 , rd_data_2 , rd_data_3 ;
wire         rd_en_0   , rd_en_1   , rd_en_2   , rd_en_3   ;
reg          rd_en_0_d1, rd_en_1_d1, rd_en_2_d1, rd_en_3_d1;
reg          rd_en_0_d2, rd_en_1_d2, rd_en_2_d2, rd_en_3_d2;
reg          ch0_rx_rstn_d1,ch1_rx_rstn_d1,ch2_rx_rstn_d1,ch3_rx_rstn_d1;
reg          ch0_tx_rstn,ch1_tx_rstn,ch2_tx_rstn,ch3_tx_rstn;
////////////////////////////////////////////////////////////////////////////////
always  @(posedge ch0_gmii_tx_clk)begin
    ch0_rx_rstn_d1 <= ch0_rx_rstn ;
    ch0_tx_rstn <= ch0_rx_rstn_d1 ;
end
always  @(posedge ch1_gmii_tx_clk)begin
    ch1_rx_rstn_d1 <= ch1_rx_rstn ;
    ch1_tx_rstn <= ch1_rx_rstn_d1 ;
end
always  @(posedge ch2_gmii_tx_clk)begin
    ch2_rx_rstn_d1 <= ch2_rx_rstn ;
    ch2_tx_rstn <= ch2_rx_rstn_d1 ;
end
always  @(posedge ch3_gmii_tx_clk)begin
    ch3_rx_rstn_d1 <= ch3_rx_rstn ;
    ch3_tx_rstn <= ch3_rx_rstn_d1 ;
end

//wr////////////////////////////////////////////////////////////////////////////
assign wr_data_0 = {ch0_gmii_rx_dv,ch0_gmii_rxd,ch0_gmii_rx_er} ;
assign wr_data_1 = {ch1_gmii_rx_dv,ch1_gmii_rxd,ch1_gmii_rx_er} ;
assign wr_data_2 = {ch2_gmii_rx_dv,ch2_gmii_rxd,ch2_gmii_rx_er} ;
assign wr_data_3 = {ch3_gmii_rx_dv,ch3_gmii_rxd,ch3_gmii_rx_er} ;
```

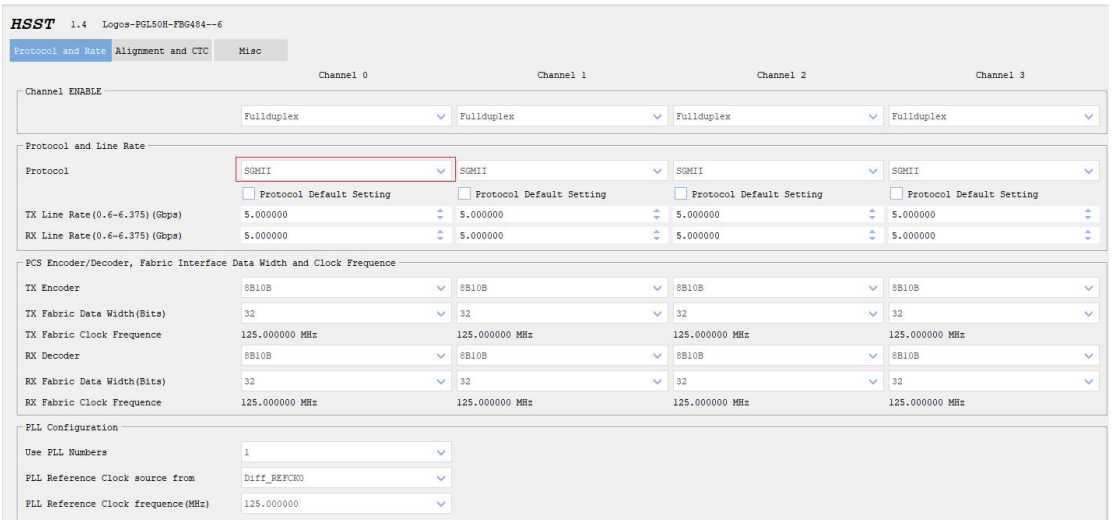
```
83.
84. assign wr_en_0   =  ch0_gmii_rx_dv ;
85. assign wr_en_1   =  ch1_gmii_rx_dv ;
86. assign wr_en_2   =  ch2_gmii_rx_dv ;
87. assign wr_en_3   =  ch3_gmii_rx_dv ;
88. //rd////////////////////////////////////////////////////////////////////////
    ////////////////////////////////////
89. assign  rd_en_0 = ~rd_empty_0 ;
90. assign  rd_en_1 = ~rd_empty_1 ;
91. assign  rd_en_2 = ~rd_empty_2 ;
92. assign  rd_en_3 = ~rd_empty_3 ;
93.
94. always  @(posedge ch0_gmii_tx_clk)begin
95.     rd_en_0_d1 <= rd_en_0    ;
96.     rd_en_0_d2 <= rd_en_0_d1 ;
97. end
98. always  @(posedge ch1_gmii_tx_clk)begin
99.     rd_en_1_d1 <= rd_en_1 ;
100.     rd_en_1_d2 <= rd_en_1_d1 ;
101. end
102. always  @(posedge ch2_gmii_tx_clk)begin
103.     rd_en_2_d1 <= rd_en_2 ;
104.     rd_en_2_d2 <= rd_en_2_d1 ;
105. end
106. always  @(posedge ch3_gmii_tx_clk)begin
107.     rd_en_3_d1 <= rd_en_3 ;
108.     rd_en_3_d2 <= rd_en_3_d1 ;
109. end
110. //ch0////////////////////////////////////////////////////////////////////////
    ////////////////////////////////////
111. fifo_0 fifo_ch0 (
112.   //wr
113.   .wr_clk       ( ch0_gmii_rx_clk     ),// input
114.   .wr_rst       (~ch0_rx_rstn         ),// input
115.   .wr_data      ( wr_data_0           ),// input [9:0]
116.   .wr_en        ( wr_en_0             ),// input
117.   .wr_full      (                     ),// output
118.   .almost_full  (                     ),// output
119.   //rd
120.   .rd_clk       ( ch0_gmii_tx_clk     ),// input
121.   .rd_rst       (~ch0_tx_rstn         ),// input
122.   .rd_data      ( rd_data_0           ),// output [9:0]
123.   .rd_en        ( rd_en_0             ),// input
124.   .rd_empty     ( rd_empty_0          ),// output
```

```
  .almost_empty (                    ) // output
);
//ch1////////////////////////////////////////////////////////////////////////////////////
fifo_0 fifo_ch1 (
  //wr
  .wr_clk       ( ch1_gmii_rx_clk    ),// input
  .wr_rst       (~ch1_rx_rstn        ),// input
  .wr_data      ( wr_data_1          ),// input [9:0]
  .wr_en        ( wr_en_1            ),// input
  .wr_full      (                    ),// output
  .almost_full  (                    ),// output
  //rd
  .rd_clk       ( ch1_gmii_tx_clk    ),// input
  .rd_rst       (~ch1_tx_rstn        ),// input
  .rd_data      ( rd_data_1          ),// output [9:0]
  .rd_en        ( rd_en_1            ),// input
  .rd_empty     ( rd_empty_1         ),// output
  .almost_empty (                    ) // output
);
//ch2////////////////////////////////////////////////////////////////////////////////////
fifo_0 fifo_ch2 (
  //wr
  .wr_clk       ( ch2_gmii_rx_clk    ),// input
  .wr_rst       (~ch2_rx_rstn        ),// input
  .wr_data      ( wr_data_2          ),// input [9:0]
  .wr_en        ( wr_en_2            ),// input
  .wr_full      (                    ),// output
  .almost_full  (                    ),// output
  //rd
  .rd_clk       ( ch2_gmii_tx_clk    ),// input
  .rd_rst       (~ch2_tx_rstn        ),// input
  .rd_data      ( rd_data_2          ),// output [9:0]
  .rd_en        ( rd_en_2            ),// input
  .rd_empty     ( rd_empty_2         ),// output
  .almost_empty (                    ) // output
);
//ch3////////////////////////////////////////////////////////////////////////////////////
fifo_0 fifo_ch3 (
  //wr
  .wr_clk       ( ch3_gmii_rx_clk    ),// input
  .wr_rst       (~ch3_rx_rstn        ),// input
```

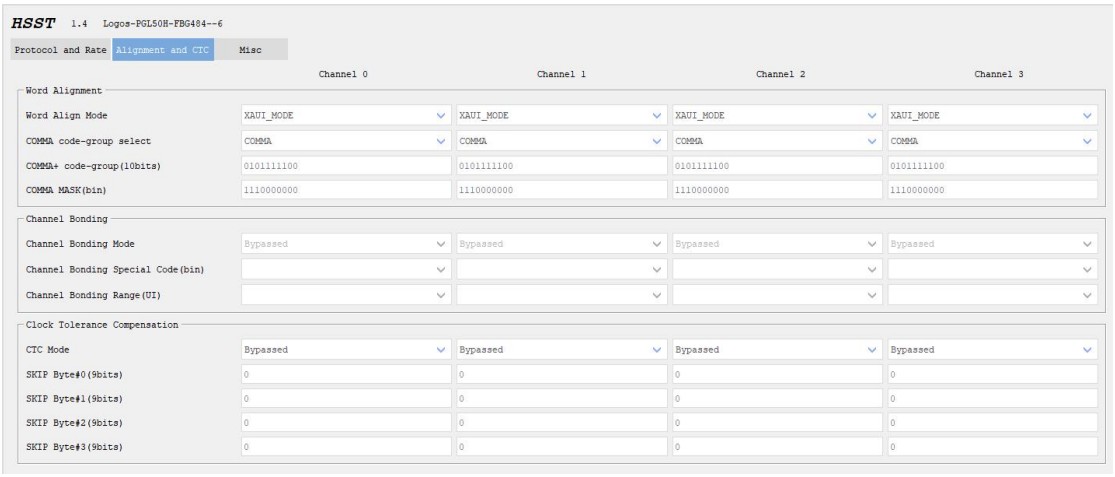
```
    .wr_data      ( wr_data_3          ),// input [9:0]
    .wr_en        ( wr_en_3            ),// input
    .wr_full      (                    ),// output
    .almost_full  (                    ),// output
    //rd
    .rd_clk       ( ch3_gmii_tx_clk    ),// input
    .rd_rst       (~ch3_tx_rstn        ),// input
    .rd_data      ( rd_data_3          ),// output [9:0]
    .rd_en        ( rd_en_3            ),// input
    .rd_empty     ( rd_empty_3         ),// output
    .almost_empty (                    ) // output
);
//out////////////////////////////////////////////////////////////////////////
//////////////////////////////////////
assign {ch0_gmii_tx_en,ch0_gmii_txd,ch0_gmii_tx_er} =  rd_en_0_d2?rd_data_0:
10'h0;
assign {ch1_gmii_tx_en,ch1_gmii_txd,ch1_gmii_tx_er} =  rd_en_1_d2?rd_data_1:
10'h0;
assign {ch2_gmii_tx_en,ch2_gmii_txd,ch2_gmii_tx_er} =  rd_en_2_d2?rd_data_2:
10'h0;
assign {ch3_gmii_tx_en,ch3_gmii_txd,ch3_gmii_tx_er} =  rd_en_3_d2?rd_data_3:
10'h0;
////////////////////////////////////////////////////////////////////////////
//////////////////////////////////////
endmodule
```

该模块主要完成数据的跨时钟域传输。一共有四路数据输入，和四路数据输出，通过 FIFO 完成数据的跨时钟域传输，该模块整体比较简单，当数据有效时就将数据写入 FIFO 之中，当读有效时，就把数据从 FIFO 读出，四路的逻辑一模一样，只需要复制四份即可，读者也可以复制出 8 份，甚至更多，或者减少成一路也可以。

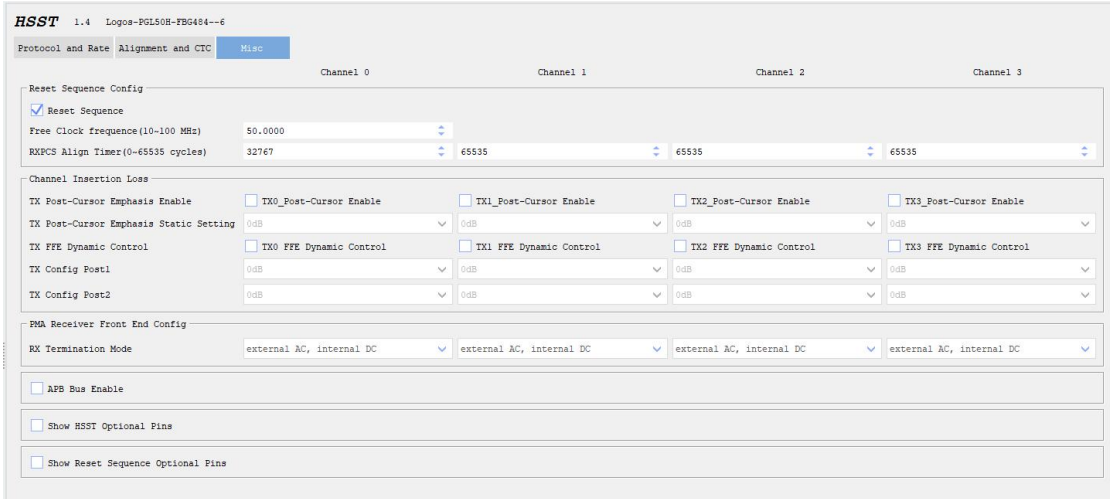
2.5 HSST IP 配置



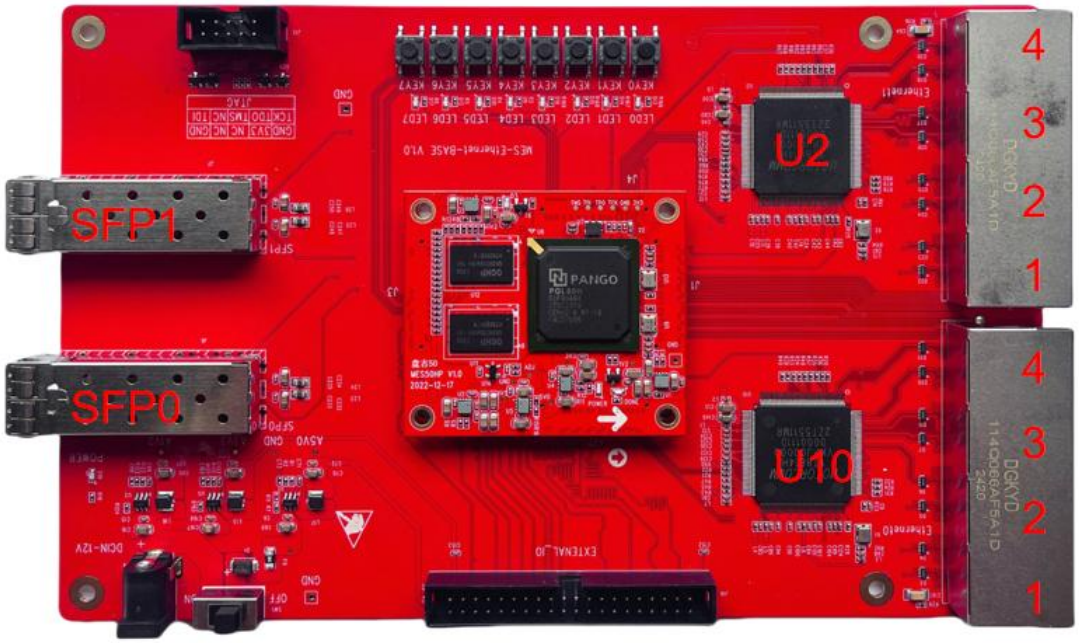
4 条 lane 的协议均选择 SGMII，参考时钟选择 0，时钟 125MHZ。



第二个选项卡的配置如上所示。

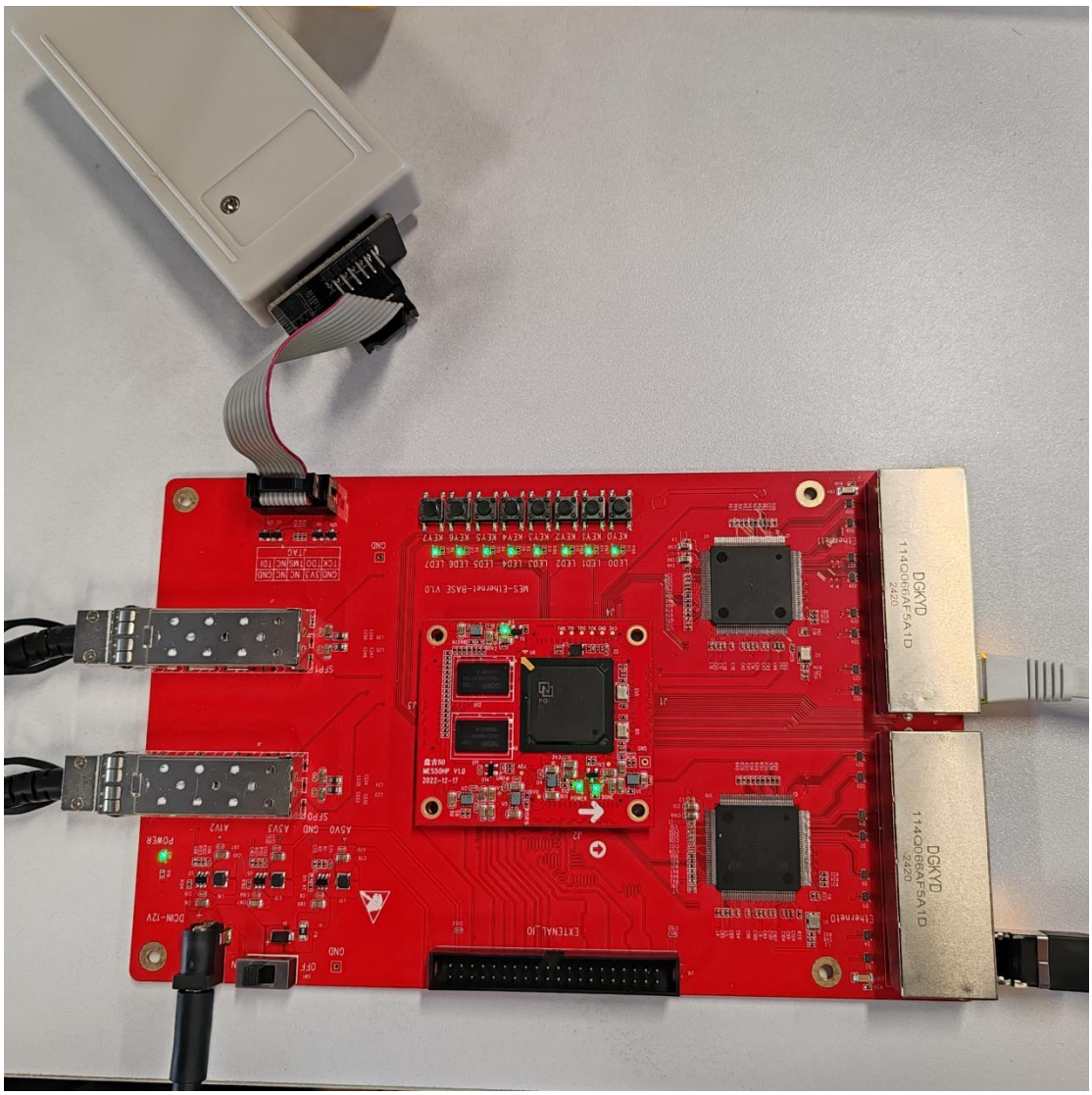


三、上板现象

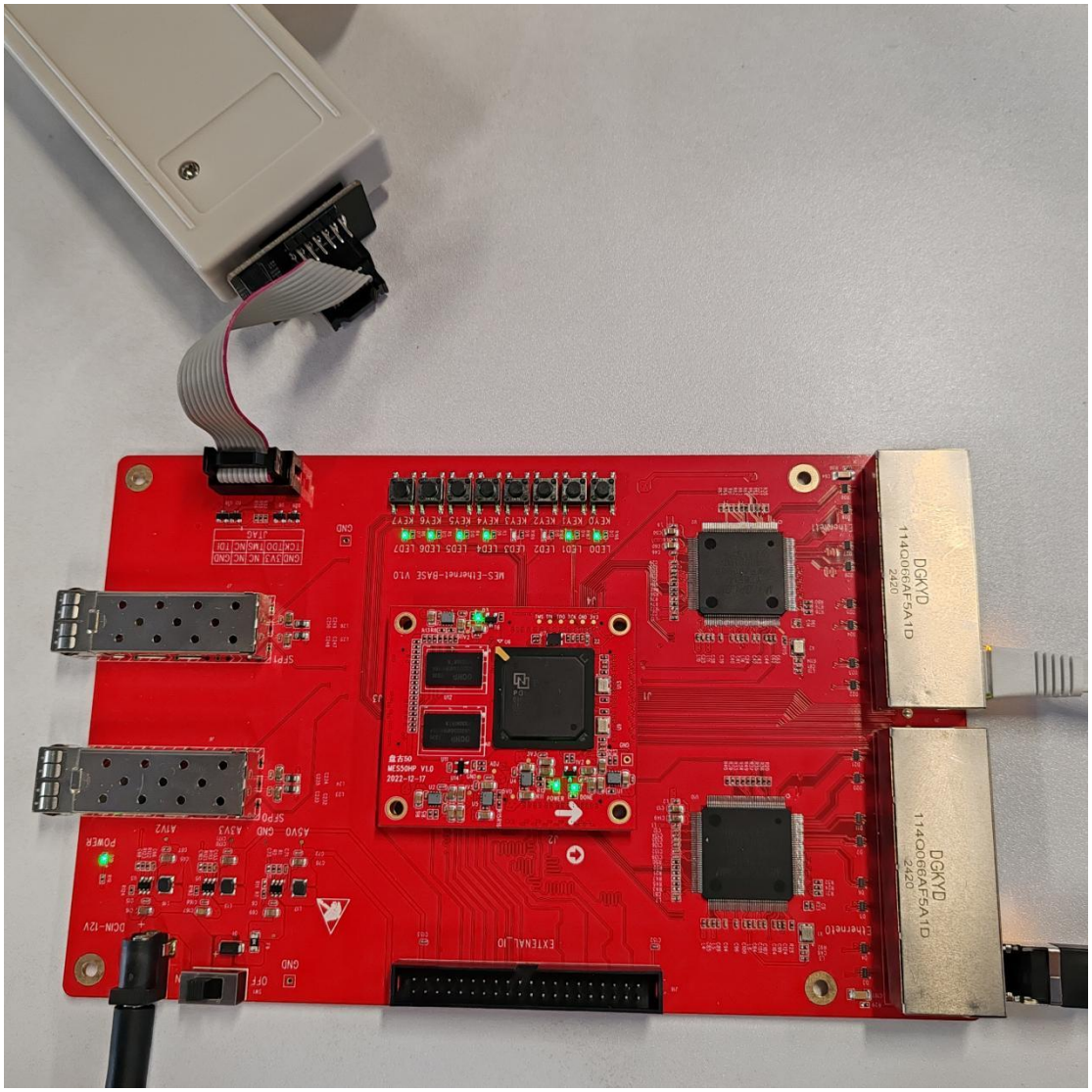


SFP1 和 SFP0 口插上光模块。八路网口，上方四路网口要和下方四路网口一一对应，例如网线插上方的网口 1，那么下方的网线也要插入网口 1 才能完成自环通信。

连接好所有线后，下载完程序后，可以看到板子上的 8 个 LED 灯全亮。表示链路全部正常。



如果 SFP 口没有插上光模块。LED2 和 LED3 不会点亮，如下所示：


DGKYD
114Q08AF5A1D
2420
DGKYD
114Q08AF5A1D
2420

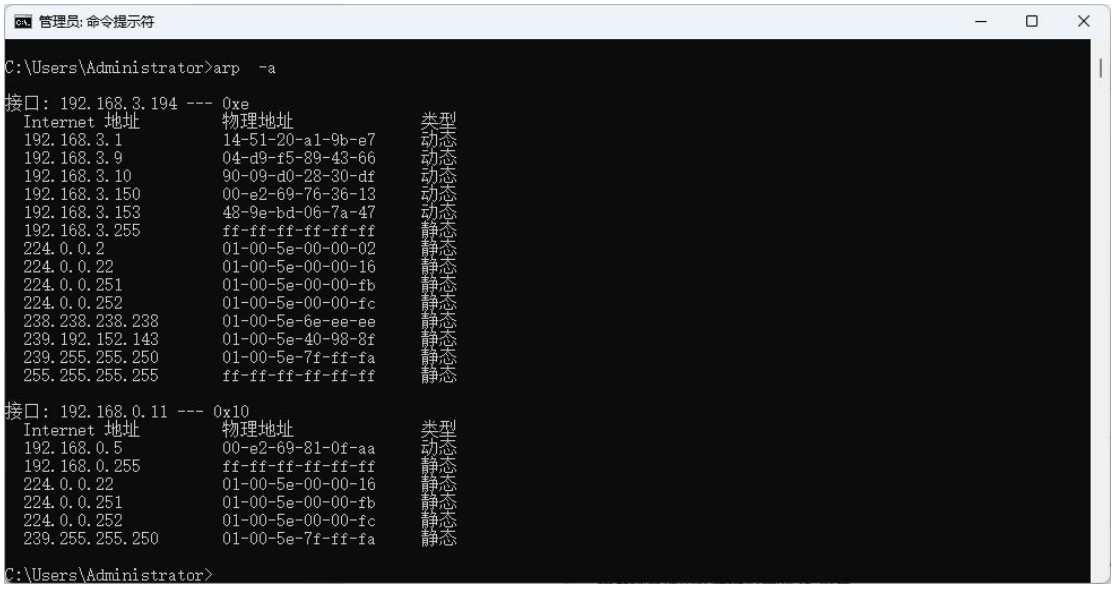
PC1 IP 设置如下(不必与我一致，保证两台 PC 的 IP 在同一网段即可)：



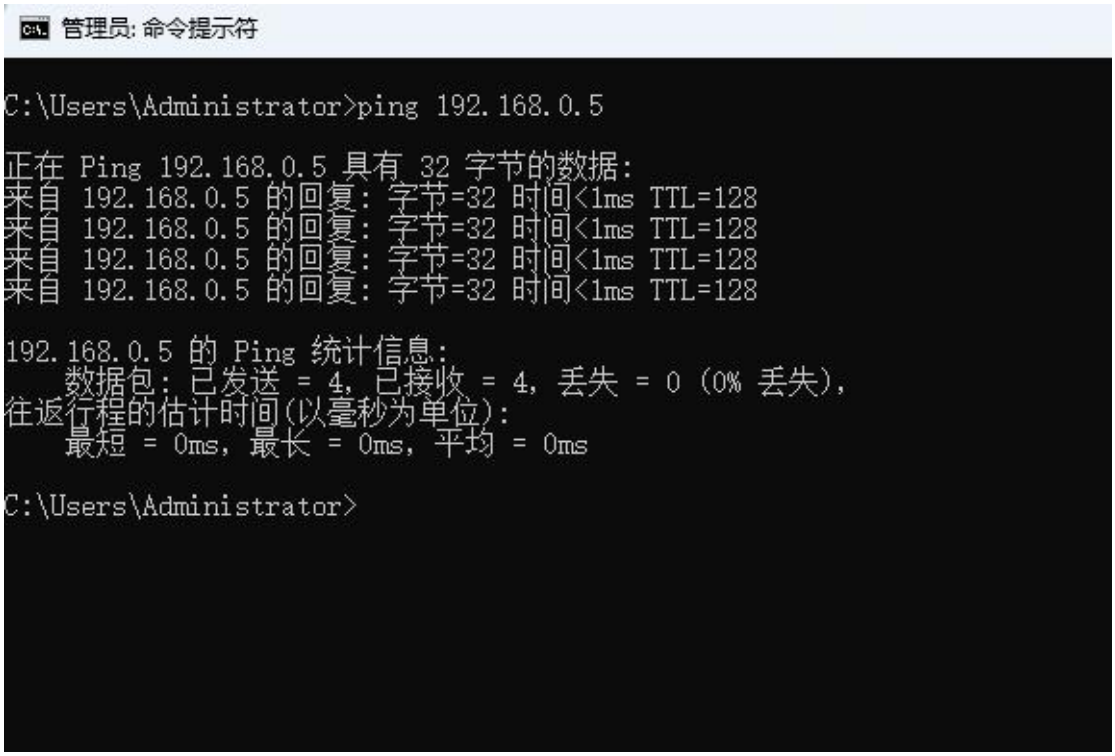
PC2 的 IP 如下：


Internet 协议版本 4 (TCP/IPv4) 属性
常规
如果网络支持此功能，则可以获取自动指派的 IP 设置。否则，你需要从网络系统管理员处获得适当的 IP 设置。
自动获得 IP 地址(O)
使用下面的 IP 地址(S):
IP 地址(I): 192 . 168 . 0 . 5
子网掩码(U): 255 . 255 . 255 . 0
默认网关(D): 192 . 168 . 0 . 1
自动获得 DNS 服务器地址(B)
使用下面的 DNS 服务器地址(E):
首选 DNS 服务器(P):
备用 DNS 服务器(A):
退出时验证设置(L)
高级(V)...
确定
取消

然后打开任一一台 PC 的 CMD，输入 ARP -A，可以看到显示出 192.168.0.5 的 IP 地址即 PC2 的 IP 地址。(192.168.3.194 是我的电脑的另一张网卡，主要看 192.168.0.11)



然后输入 ping 192.168.0.11 开始 ping 第二台电脑，如下所示：



最后，可以 PING 通，则表示实验成功。